

wrapper for TensorFlow, which can be installed using *Pkg.add*, as shown below:

```
Pkg.add("TensorFlow")
```

Using the GPU for efficient processing is an important attribute of machine learning approaches. To activate the GPU support, the environment variable *TF_USE_GPU* needs to be set to 1. To enable GPU usage, CUDA 7.5 and *cuDNN* are needed.

```
ENV["TF_USE_GPU"] = "1"
Pkg.build("TensorFlow")
```

It has to be noted that the TensorFlow API is huge, and all the functionalities are not wrapped at present. Some of the currently available functionalities are listed below:

- Unary and binary mathematical functions
 - Commonly used neural network operations. This covers convolutions, recurrent neural networks, etc
 - Fundamental image-loading and resizing operations
- The following are not currently wrapped:
- Control flow operations
 - Distributed graph execution
 - PyBoard graph visualisation

A code fragment to classify MNIST digits with TensorFlow is provided in this section (<https://malmaud.github.io/tfdocs/tutorial/>).

1. The TensorFlow session is built as shown below:

```
using TensorFlow
sess = Session()
```

2. A Softmax Regression Model is built as follows:

```
x = placeholder(Float32)
y = placeholder(Float32)
W = Variable(zeros([784, 10]))
b = Variable(zeros([10]))
run(sess, initialize_all_variables())
y = nn.softmax(x*W + b)
cross_entropy = reduce_mean(-reduce_sum(y_* log(y),
reduction_indices=[2]))
```

3. Training of the model is shown below:

```
train_step = train.minimize(train.
GradientDescentOptimizer(.00001), cross_entropy)
for i in 1:1000
    batch = next_batch(loader, 100)
    run(sess, train_step, Dict{x=>batch[1], y=>batch[2]})
end
```

4. The model evaluation is done using the following code snippet:

```
correct_prediction = indmax(y, 2) == indmax(y_, 2)
accuracy=reduce_mean(cast(correct_prediction, Float32))
testx, testy = load_test_set()
```

```
println(run(sess, accuracy, Dict{x=>testx, y=>testy}))
```

MXNet

MXNet is an active deep learning package available in Julia. It facilitates efficient GPU handling for optimal results.

MXNet.jl is the Julia version of *dmlc/mxnet* (<https://github.com/dmlc/mxnet>). The major features of *MXNet.jl* are:

- It enables efficient tensor computation; it can efficiently handle multiple computing devices, GPUs and distributed nodes.
- It facilitates effective manipulation of deep learning models.
- A simple three-layer MLP can be defined as follows:

```
using MXNet
data = mx.Variable{:data}

fc1 = mx.FullyConnected(data = data, name=:fc1, num_
hidden=128)
act1 = mx.Activation(data = fc1, name=:relu1, act_type=:relu)
fc2 = mx.FullyConnected(data = act1, name=:fc2, num_
hidden=64)
act2 = mx.Activation(data = fc2, name=:relu2, act_type=:relu)
fc3 = mx.FullyConnected(data = act2, name=:fc3, num_
hidden=10)
```

The aforementioned code fragment forms a feedforward chain. The recognition of digits 0 to 9 is handled with 10 output classes. A complete tutorial sequence is available at <http://dmlc.ml/MXNet.jl/latest/tutorial/mnist/#Convolutional-Neural-Networks-1>.

Machine learning incorporates a large spectrum of approaches. This article is just the tip of the iceberg. If you are interested in learning more about the topic, then the following link has pointers to plenty of resources in one place: <https://github.com/josephmisiti/awesome-machine-learning>. 

References

- [1] <https://github.com/dmlc/mxnet>
- [2] <http://scikitlearnjl.readthedocs.io/en/latest/quickstart/>
- [3] <https://github.com/josephmisiti/awesome-machine-learning>
- [4] <https://github.com/malmaud/TensorFlow.jl>

By: Dr K.S. Kuppusamy

The author is assistant professor of computer science at the Pondicherry Central University. He has 11+ years of experience in academia, research and industry. He can be reached via mail at kskuppua@gmail.com.